Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Арифметические операции с полиномами»

Выполнил:
студент группы 3824Б1ФИ2
Ципин Д. Д.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 6
3. Руководство программиста 8
3.1. Описание структуры программы 8
3.2. Описание структур данных 8
3.3. Описание алгоритмов 9
4. Результаты экспериментов и тестирования 11
Заключение 12
Литература 13

Введение

Алгебраические вычисления над полиномами являются одной из фундаментальных задач в области компьютерной алгебры, символьных вычислений и численных методов. В программировании выбор структуры данных для представления полиномов напрямую влияет на эффективность использования памяти и скорость выполнения арифметических операций.

Статичные структуры данных часто оказываются неэффективными для хранения полиномов, особенно в случаях многочленов, где большинство коэффициентов равны нулю. В данной работе рассматривается реализация полиномов от трех переменных с использованием динамической структуры данных — двусвязного списка. Такой подход позволяет хранить только ненулевые мономы, обеспечивая эффективное управление памятью и реализацию алгоритмов сложения и умножения через манипуляции с узлами списка.

1. Постановка задачи

Целью данной работы является разработка программы на языке C++, выполняющего арифметические операции над полиномами от трех переменных – X, Y и Z.

К реализуемым операциям относятся:

1. Сложение двух полиномов.

2. Вычитание полиномов.

3. Умножение полинома на вещественную константу.

4. Перемножение двух полиномов.

Технические требования и особенности реализации:

- **Хранение данных:** Полином должен быть представлен в виде упорядоченного списка мономов с ненулевыми коэффициентами. В данной работе выбран двусвязный список.
- **Представление степеней:** Для оптимизации памяти степень монома должна храниться в «свернутом» виде как трехзначное целое число: разряд сотен отвечает за степень X, десятков — за Y, единиц — за Z. Каждая степень ограничена от 0 до 9.
- **Алгоритм сложения:** Сложение должно выполняться по принципу слияния двух упорядоченных списков, что обеспечивает линейную сложность.
- **Алгоритм вычитания:** Вычитание реализовано через сложение первого полинома с результатом умножения второго полинома на константу −1.
- **Корректность хранения:** Программа должна автоматически приводить подобные слагаемые и исключать из списка мономы, коэффициенты которых стали равны нулю в ходе вычислений.
- **Контроль ошибок:** При перемножении полиномов необходимо реализовать проверку на переполнение степеней (сумма степеней каждой переменной не должна превышать 9).

- **Ввод данных:** Программа должна корректно обрабатывать ввод пользователя, даже если мономы вводятся в неупорядоченном порядке.
- **Тестирование:** Работоспособность всех модулей должна быть подтверждена набором модульных тестов с использованием фреймворка Google Test.

2. Руководство пользователя

Программа представляет собой консольное приложение с интерактивным текстовым меню. Взаимодействие осуществляется путем ввода номера команды и последующего следования инструкциям на экране.

2.1. Формат представления данных

Полиномы состоят из мономов, каждый из которых характеризуется вещественным коэффициентом и целой степенью.

- **Степень монома** вводится как трехзначное число:
  - Первая цифра (сотни) — степень переменной **x**.
  - Вторая цифра (десятки) — степень переменной **y**.
  - Третья цифра (единицы) — степень переменной **z**.
- *Пример:* Ввод 2.5 123 соответствует члену полинома $2.5 \cdot x^1 y^2 z^3$.

2.2. Запуск и работа с меню

После запуска программы пользователю выводится главное меню («**ACTIONS MENU**»):

1. **Enter Polynom 1 (P1)** — Ввод первого слагаемого/множимого.
2. **Enter Polynom 2 (P2)** — Ввод второго слагаемого/множителя.
3. **Show the current polynomials** — Вывод на экран текущих значений P1 и P2.
4. **Add (P1 + P2)** — Вычисление суммы двух полиномов.
5. **Div (P1 - P2)** — Вычисление разности (P1 минус P2).
6. **Mul (P1 * P2)** — Перемножение двух полиномов.
7. **Mul P1 by a number** — Умножение первого полинома на введенное число.
8. **Exit** — Завершение работы программы.

2.3. Инструкция по вводу полинома (Команды 1 и 2)

При выборе пунктов 1 или 2 программа переходит в режим ожидания ввода мономов.

1. Введите пару чисел через пробел: <коэффициент> <степень>.
2. Нажмите **Enter**.
3. Продолжайте ввод следующих мономов. Мономы можно вводить в любом порядке.

4. Для завершения ввода полинома нажмите **Enter** на пустой строке.

2.4. Обработка ошибок

- Если при умножении полиномов суммарная степень одной из переменных превысит **9**, программа выведет сообщение об ошибке: *An error has occurred: degree overflow*.
- При вводе некорректных символов (букв вместо чисел) программа выдаст предупреждение: *Input error. Enter a number* и попросит повторить ввод.

2.5. Пример использования

Для вычисления выражения $(x^2z^2+2xz)+(3x^2z^2)$:

1. Выбрать пункт **1**. Ввести 1 202, затем 2 101, нажать **Enter** на пустой строке.
2. Выбрать пункт **2**. Ввести 3 202, нажать **Enter** на пустой строке.
3. Выбрать пункт **4**. Программа выведет результат: 4 202, 2 101.

3. Руководство программиста

3.1. Описание структуры программы

Программа построена по модульному принципу и состоит из следующих компонентов:

1. **list.h**: Содержит реализацию низкоуровневой структуры данных. Включает классы **Node** (узел списка) и **List** (двусвязный список). Здесь находится логика управления памятью и базовые операции со списком.
2. **polynomial.h**: Содержит класс **Polynom**, который является надстройкой над списком. Реализует математическую логику: обработка ввода, форматированный вывод и перегрузку арифметических операторов.
3. **Lab1.cpp**: Главный модуль приложения. Содержит функцию **main**, реализацию меню управления и обработку исключений.
4. **tests.cpp**: Модуль тестирования на базе Google Test, проверяющий корректность всех компонентов.

3.2. Описание структур данных

Основными структурами данных являются три взаимосвязанных класса:

- **Класс Node (Узел)**:
  - **power (int):** Свернутая степень (XYZ).
  - **coeff (double):** Коэффициент монома.
  - **next, prev**: Указатели на соседние элементы.
  - ***Особенности***: Перегружены операторы сравнения (>, <, ==)
- **Класс List (Двусвязный список)**:
  - **head, tail**: Указатели на начало и конец списка.
  - **push_back(co, pow):** Метод вставки с автоматическим поиском позиции для сохранения порядка и приведением подобных слагаемых.
  - **operator= и конструктор копирования**: Реализуют глубокое копирование для предотвращения ошибок при работе с динамической памятью.
  - **print()**: вывод списка в терминал формата <коэффициент> <степень>

- **Класс Polynom (Полином)**:
  - **poly (List)**: объект списка для хранения мономов.
  - **print():** дублирует метод print() из класса **List**
  - **operator +/-/*:** арифметические операции к полиномам.
  - ***Особенности***: Все арифметические операции (+, -, *) возвращают новый объект Polynom, не изменяя операндов.

3.3. Описание алгоритмов

1. Алгоритм сравнения мономов (сортировка): Для соблюдения математического порядка реализовано сравнение наборов степеней. Алгоритм извлекает разряды числа **power**, сортирует их по убыванию внутри одного монома и сравнивает полученные наборы. Если наборы идентичны, применяется лексикографическое сравнение (по значению power). Это гарантирует приоритет мономов с более высокими степенями отдельных переменных (например, y^3 будет стоять выше, чем x^2z^2 или x^3z будет стоять выше xz^3).

2. Алгоритм сложения (Слияние): Сложение реализовано как слияние двух упорядоченных списков (сложность O(N+M)).

- Если степени равны — коэффициенты складываются, результат заносится в новый список.
- Если степень текущего монома P1 «старше» — он переносится в результат, указатель P1 сдвигается.
- Аналогично для P2.
- В конце выполняется **remove_zeros()** для удаления элементов с нулевым результатом.

3. Алгоритм умножения: Реализовано перемножение «каждый на каждого». Для каждой пары мономов из P1 и P2:

1. Вызывается проверка переполнения степеней (поразрядная сумма не должна превышать 9).
2. Коэффициенты перемножаются, степени складываются.
3. Результат добавляется в новый список через **poly.push_back()**, который автоматически находит нужное место и суммирует коэффициенты, если такая степень уже была получена от предыдущих пар.

4. Удаление нулевых коэффициентов: Метод **remove_zeros()** проходит по списку и удаляет узлы, чей коэффициент равен 0.

4. Результаты экспериментов и тестирования

Для проверки корректности работы программы было проведено комплексное тестирование, включающее модульные тесты (Google Tests).

Тестовое покрытие включает 13 сценариев, разделенных на три логические группы:

1. **Тестирование логики сравнения (NodeTest)**:
   - Проверено корректное сравнение степеней. Тест *MoreLessEqual* подтвердил, что моном со степенью 033 (максимальная степень 3) признается «старшим» по отношению к 202 (максимальная степень 2), что соответствует математическому порядку.
   - Проверена лексикографическая сортировка при равенстве максимальных степеней (например, 210 > 120).
2. **Тестирование структуры данных (ListTest)**:
   - **Управление памятью**: Тесты *CopyConstructor* и *Assignment* подтвердили реализацию глубокого копирования. Изменение данных в копии списка не влияет на оригинал.
   - **Целостность данных**: Тест *PushBackSortedInsertion* подтвердил, что при вводе мономов в произвольном порядке список всегда остается упорядоченным.
   - **Приведение подобных**: Тест *PushBackSameCoefficients* показал, что мономы с одинаковой степенью успешно суммируются в один узел.
3. **Тестирование арифметики полиномов (PolynomTest)**:
   - **Сложение и вычитание**: Тест *SubtractionToZero* подтвердил, что при получении нулевого результата (например, $10x^5 - 10x^5$) моном полностью удаляется из памяти, а голова списка становится *nullptr*.
   - **Умножение**: Проверено распределительное свойство умножения (каждый на каждого). Тест *PolynomialMultiplication* успешно вычислил формулу разности квадратов: $(x+1)(x-1)=x^2-1$.
   - **Обработка исключений**: Тест *MultiplicationDegreeOverflow* подтвердил генерацию исключения *std::runtime_error* при попытке получить степень переменной выше 9 (например, $x^9 \cdot x^1$).

Заключение

В ходе выполнения данной лабораторной работы был разработан программный комплекс для выполнения арифметических операций над полиномами от трех переменных. В процессе реализации были решены ключевые задачи по организации динамических структур данных и оптимизации алгоритмов обработки многочленов.

Основные результаты работы:

1. **Выбор структуры данных:** Реализован двусвязный динамический список, который позволил эффективно представлять разреженные полиномы, минимизируя затраты памяти за счет хранения только ненулевых мономов.
2. **Математическая логика:** Реализованы алгоритмы сложения (методом слияния), вычитания и перемножения полиномов. Система «свернутого» представления степеней (XYZ) позволила эффективно проверять условия переполнения и выполнять поразрядные операции над показателями.
3. **Автоматизация упорядочивания:** Разработана уникальная логика сравнения мономов, учитывающая приоритет максимальной степени переменной. Это обеспечило автоматическую сортировку полиномов при вводе и выполнении операций, соответствующую математическим стандартам.
4. **Тестирование:** С помощью фреймворка Google Test было проведено всестороннее тестирование программы. Тесты подтвердили корректность приведения подобных слагаемых, удаления нулевых коэффициентов и обработки исключительных ситуаций (переполнение степеней).
5. **Интерфейс:** Создано консольное приложение с интерактивным меню, обеспечивающее удобный ввод и визуализацию результатов вычислений.

Таким образом, разработанная программа полностью соответствует поставленной задаче.

Литература

1. Сайт cppreference: https://en.cppreference.com/